

Examen UE architecture et système 2

Licence informatique 2021/2022, première session

Stéphane Rubini

Durée : 3 heures

Sans document

Barème indicatif : ex 1-4 points, ex 2-3 points, ex 3-3 points, ex 4-10 points

Exercice 1 : questions de cours

Pour cet exercice, la qualité de la rédaction des réponses, en plus de leur exactitude, sera un critère d'évaluation.

1. Quand quels cas est-il utile d'utiliser un mécanisme "producteurs/consommateurs" pour implanter une communication entre processus ? Citer une mise en œuvre courante de ce mécanisme dans le "monde UNIX".
2. Quelles sont les différences entre un thread et un processus ?
3. Un processus est programmé de manière à émettre un signal Unix SIGUSR1 à chaque fois qu'il détecte un événement donné. Est-il possible pour un autre processus de comptabiliser le nombre d'événements détectés par le premier processus à partir du nombre de signaux SIGUSR1 reçus ? Si non, expliquez pourquoi ?
4. Quelle est la différence entre le swap, la mémoire paginée et la mémoire virtuelle ?
5. Dans la stratégie d'ordonnancement des threads sous Linux, que représente la notion d'affinité ?

Exercice 2 : ordonnancement

Soit le modèle de tâches suivant :

Tâche	Instant d'arrivée	Temps de service
A	0	5
B	1	4
C	4	2
D	2	5
E	5	1

Représenter l'ordonnancement des tâches pour les stratégies d'ordonnancement Batch "*Shortest Process Next*" et tourniquet (quantum=1).

Exercice 3 : synchronisation

On essaie d'implanter un **sémaphore compteur** à l'aide des fonctions suivantes.

```
struct sem {
    int c;
    pthread_mutex_t mutex;
    pthread_mutex_t stop;
};

void init(struct sem *s, int val_init) {
    s->c=val_init;
    pthread_mutex_init(&s->mutex, NULL);
    pthread_mutex_init(&s->stop, NULL);
    pthread_mutex_lock(&s->stop);
}

void P(struct sem *s) {
    pthread_mutex_lock(&s->mutex);
    s->c = s->c - 1;
    if ( s->c<0 ) pthread_mutex_lock(&s->stop);
    pthread_mutex_unlock(&s->mutex);
}

void V(struct sem *s) {
    pthread_mutex_lock(&s->mutex);
    s->c = s->c + 1;
    if ( s->c==0 ) pthread_mutex_unlock(&s->stop);
    pthread_mutex_unlock(&s->mutex);
}
```

La valeur du compteur est enregistrée dans le champ `c` de la structure `sem`.

1. Ces fonctions ne sont pas correctes. Expliquez pourquoi ?
2. Proposer une mise en œuvre du **sémaphore compteur** avec des "mutexes" et des variables de condition.